Data_preprocessing

October 26, 2025

0.0.1 Data Preprocessing

[ ]:
```
import pandas as pd
import numpy as np
df = pd.read_csv('../../../data/earthquakes_data.csv')
df.sample(100)
```

[ ]:

	Unnamed: 0	time	latitude	longitude	depth	\
42337	42337	1990-04-04T09:07:27.230Z	-0.7880	122.3230	32.6	
25384	25384	1979-05-28T00:02:32.100Z	-17.5470	-175.2010	275.0	
108970	108970	2025-07-24T13:08:19.728Z	-2.0671	120.5841	35.0	
18297	18297	1974-12-26T21:36:27.200Z	12.9270	125.1700	33.0	
16619	16619	1973-09-12T06:59:54.300Z	73.3020	55.1610	0.0	
…	…	…	…	…	…	
19826	19826	1975-12-14T22:14:40.000Z	13.0270	125.7850	33.0	
98237	98237	2023-09-03T21:29:58.745Z	6.8276	147.9695	10.0	
10879	10879	1964-02-05T11:35:23.640Z	-19.6810	-179.6970	460.0	
63778	63778	2005-04-01T07:20:09.010Z	14.3220	-92.9610	22.6	
53581	53581	1997-11-18T15:23:31.830Z	37.3530	21.1720	33.0	

	mag	magType	nst	gap	dmin	…	updated	\
42337	5.10	mw	NaN	NaN	NaN	…	2022-04-28T19:51:05.633Z	
25384	5.30	mb	NaN	NaN	NaN	…	2016-11-09T22:37:36.757Z	
108970	5.60	mww	49.0	68.0	0.555	…	2025-08-11T14:50:38.450Z	
18297	5.10	mb	NaN	NaN	NaN	…	2014-11-06T23:21:28.008Z	
16619	6.80	mb	NaN	NaN	NaN	…	2015-06-16T17:17:30.808Z	
…	…	…	…	…	…	…	…	
19826	5.20	mb	NaN	NaN	NaN	…	2014-11-06T23:21:32.932Z	
98237	5.50	mww	121.0	33.0	7.379	…	2023-09-04T21:34:50.580Z	
10879	5.51	mw	NaN	NaN	NaN	…	2022-04-27T00:15:33.641Z	
63778	5.10	mwc	166.0	71.7	NaN	…	2016-11-10T01:03:53.962Z	
53581	5.10	mb	NaN	NaN	NaN	…	2014-11-07T01:03:55.670Z	

	place	type	\
42337	54 km WNW of Luwuk, Indonesia	earthquake	
25384	177 km NW of Neiafu, Tonga	earthquake	
108970	60 km NNE of Masamba, Indonesia	earthquake	
18297	32 km NNE of Cabodiongan, Philippines	earthquake	

```
16619                          Novaya Zemlya, Russia  nuclear explosion
…                                                …                …
19826              82 km NE of Cabatuan, Philippines         earthquake
98237   State of Yap, Federated States of Micronesia         earthquake
10879                     206 km SSE of Levuka, Fiji         earthquake
63778              72 km SW of Puerto Madero, Mexico         earthquake
53581                   41 km SW of Epitálio, Greece         earthquake

       horizontalError depthError magError  magNst    status  locationSource  \
42337              NaN      7.200      NaN     NaN  reviewed              us
25384              NaN        NaN      NaN     NaN  reviewed              us
108970            3.60      1.892    0.098    10.0  reviewed              us
18297              NaN        NaN      NaN     NaN  reviewed              us
16619              NaN        NaN      NaN     NaN  reviewed              us
…                  …          …        …       …         …              …
19826              NaN        NaN      NaN     NaN  reviewed              us
98237             9.02      1.852    0.083    14.0  reviewed              us
10879              NaN      8.300    0.200     NaN  reviewed          iscgem
63778              NaN        NaN      NaN     NaN  reviewed              us
53581              NaN        NaN      NaN    75.0  reviewed              us

        magSource
42337         hrv
25384          us
108970         us
18297          us
16619          us
…             …
19826          us
98237          us
10879      iscgem
63778         hrv
53581          us

[100 rows x 23 columns]
```

[15]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 110133 entries, 0 to 110132
Data columns (total 23 columns):
 #   Column           Non-Null Count   Dtype
---  ------           --------------   -----
 0   Unnamed: 0       110133 non-null  int64
 1   time             110133 non-null  object
 2   latitude         110133 non-null  float64
 3   longitude        110133 non-null  float64
 4   depth            109848 non-null  float64
```

```
 5   mag              110133 non-null  float64
 6   magType          110133 non-null  object
 7   nst              39492 non-null   float64
 8   gap              49783 non-null   float64
 9   dmin             29839 non-null   float64
 10  rms              81386 non-null   float64
 11  net              110133 non-null  object
 12  id               110133 non-null  object
 13  updated          110133 non-null  object
 14  place            109242 non-null  object
 15  type             110133 non-null  object
 16  horizontalError  28459 non-null   float64
 17  depthError       60418 non-null   float64
 18  magError         43072 non-null   float64
 19  magNst           50108 non-null   float64
 20  status           110133 non-null  object
 21  locationSource   110133 non-null  object
 22  magSource        110133 non-null  object
dtypes: float64(12), int64(1), object(10)
memory usage: 19.3+ MB
```

[16]:
```
df.columns
```

[16]:
```
Index(['Unnamed: 0', 'time', 'latitude', 'longitude', 'depth', 'mag',
       'magType', 'nst', 'gap', 'dmin', 'rms', 'net', 'id', 'updated', 'place',
       'type', 'horizontalError', 'depthError', 'magError', 'magNst', 'status',
       'locationSource', 'magSource'],
      dtype='object')
```

[17]:
```
df.isnull().sum()
```

[17]:
```
Unnamed: 0            0
time                  0
latitude              0
longitude             0
depth               285
mag                   0
magType               0
nst               70641
gap               60350
dmin              80294
rms               28747
net                   0
id                    0
updated               0
place               891
type                  0
horizontalError   81674
```

```
depthError         49715
magError           67061
magNst             60025
status                 0
locationSource         0
magSource              0
dtype: int64
```

[18]:
```
df.shape
```

[18]:
```
(110133, 23)
```

[19]:
```
round(df.isnull().sum()*100/df.shape[0],2)
```

[19]:
```
Unnamed: 0          0.00
time                0.00
latitude            0.00
longitude           0.00
depth               0.26
mag                 0.00
magType             0.00
nst                64.14
gap                54.80
dmin               72.91
rms                26.10
net                 0.00
id                  0.00
updated             0.00
place               0.81
type                0.00
horizontalError    74.16
depthError         45.14
magError           60.89
magNst             54.50
status              0.00
locationSource      0.00
magSource           0.00
dtype: float64
```

[20]:
```
round(df.isnull().sum()*100/df.shape[0],2)
```

[20]:
```
Unnamed: 0          0.00
time                0.00
latitude            0.00
longitude           0.00
depth               0.26
mag                 0.00
```

```
magType             0.00
nst                64.14
gap                54.80
dmin               72.91
rms                26.10
net                 0.00
id                  0.00
updated             0.00
place               0.81
type                0.00
horizontalError    74.16
depthError         45.14
magError           60.89
magNst             54.50
status              0.00
locationSource      0.00
magSource           0.00
dtype: float64
```

[21]:
```python
# Display missing value percentage
missing_percent = round(df.isnull().sum() * 100 / len(df), 2)
print(missing_percent)

# Automatically detect and drop columns with >30% missing values
threshold = 30
cols_to_drop = missing_percent[missing_percent > threshold].index.tolist()
print(f"Columns dropped due to missing >{threshold}%:\n{cols_to_drop}\n")

df.drop(columns=cols_to_drop, inplace=True, errors='ignore')

# Confirm shape after dropping
print(f"Data shape after dropping columns: {df.shape}")
```

```
Unnamed: 0          0.00
time                0.00
latitude            0.00
longitude           0.00
depth               0.26
mag                 0.00
magType             0.00
nst                64.14
gap                54.80
dmin               72.91
rms                26.10
net                 0.00
id                  0.00
updated             0.00
place               0.81
```

```
type                0.00
horizontalError    74.16
depthError         45.14
magError           60.89
magNst             54.50
status              0.00
locationSource      0.00
magSource           0.00
dtype: float64
Columns dropped due to missing >30%:
['nst', 'gap', 'dmin', 'horizontalError', 'depthError', 'magError', 'magNst']

Data shape after dropping columns: (110133, 16)
```

[22]:
```
# Check remaining missing values
round(df.isnull().sum()*100/df.shape[0],2)
```

[22]:
```
Unnamed: 0         0.00
time               0.00
latitude           0.00
longitude          0.00
depth              0.26
mag                0.00
magType            0.00
rms               26.10
net                0.00
id                 0.00
updated            0.00
place              0.81
type               0.00
status             0.00
locationSource     0.00
magSource          0.00
dtype: float64
```

[23]:
```
# Drop specific ID-like columns
id_like_cols = ['Unnamed: 0', 'id', 'net', 'locationSource', 'magSource']
df.drop(columns=id_like_cols, inplace=True, errors='ignore')
```

[24]:
```
# Drop rows where 'depth' is missing (NaN)
df = df.dropna(subset=['depth'])

# Confirm rows dropped
print(f"Data shape after dropping rows with missing depth: {df.shape}")
```

```
Data shape after dropping rows with missing depth: (109848, 11)
```

[25]:
```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Select features that likely correlate with 'rms'
predictors = ['mag', 'depth', 'nst', 'gap', 'dmin']
available_features = [f for f in predictors if f in df.columns]

# Separate rows with known and unknown RMS values
df_known = df[df['rms'].notnull()]
df_missing = df[df['rms'].isnull()]

# Keep only predictors that have minimal missingness
df_known = df_known.dropna(subset=available_features)
df_missing = df_missing.dropna(subset=available_features)

# If enough known data, train regression model
if not df_known.empty and not df_missing.empty:
    X_train = df_known[available_features]
    y_train = df_known['rms']

    model = LinearRegression()
    model.fit(X_train, y_train)

    # Predict missing RMS values
    df.loc[df['rms'].isnull(), 'rms'] = model.
 ↪predict(df_missing[available_features])

print(f"Missing RMS after regression imputation: {df['rms'].isnull().sum()}")
```

```
Missing RMS after regression imputation: 0
```

[26]:
```
# Check remaining missing values
round(df.isnull().sum()*100/df.shape[0],2)
```

[26]:
```
time         0.00
latitude     0.00
longitude    0.00
depth        0.00
mag          0.00
magType      0.00
rms          0.00
updated      0.00
place        0.81
type         0.00
status       0.00
dtype: float64
```

[27]:
```
# Drop 'place' and 'updated' columns as they are not needed
df.drop(columns=['place', 'updated'], axis=1, inplace=True)
```

[28]:
```
# Convert single 'time' column to datetime
df['Datetime'] = pd.to_datetime(df['time'], errors='coerce')

# Extract temporal features
df['Year'] = df['Datetime'].dt.year
df['Month'] = df['Datetime'].dt.month
df['Day'] = df['Datetime'].dt.day
df['Hour'] = df['Datetime'].dt.hour
df['Minute'] = df['Datetime'].dt.minute
df['Second'] = df['Datetime'].dt.second
df['DayOfWeek'] = df['Datetime'].dt.dayofweek  # Monday=0, Sunday=6

# Cyclical encoding for Month, Hour, DayOfWeek
# df['Month_sin'] = np.sin(2 * np.pi * df['Month'] / 12)
# df['Month_cos'] = np.cos(2 * np.pi * df['Month'] / 12)
# df['Hour_sin'] = np.sin(2 * np.pi * df['Hour'] / 24)
# df['Hour_cos'] = np.cos(2 * np.pi * df['Hour'] / 24)
# df['DayOfWeek_sin'] = np.sin(2 * np.pi * df['DayOfWeek'] / 7)
# df['DayOfWeek_cos'] = np.cos(2 * np.pi * df['DayOfWeek'] / 7)

# # Drop original 'time' and intermediate columns
# df.drop(['time', 'Datetime', 'Month', 'Hour', 'DayOfWeek'], axis=1,␣
  ↪inplace=True)
df.drop(['time', 'Datetime'], axis=1, inplace=True)

# Check the transformed dataframe
df.head()
```

[28]:

	latitude	longitude	depth	mag	magType	rms	type	status	\
16	41.758	23.249	15.0	7.02	mw	1.032335	earthquake	reviewed	
17	41.802	23.108	15.0	6.84	mw	1.023418	earthquake	reviewed	
18	52.763	160.277	30.0	7.70	mw	1.065273	earthquake	reviewed	
19	51.424	161.638	15.0	7.50	mw	1.056114	earthquake	reviewed	
20	30.684	100.608	15.0	7.09	mw	1.035803	earthquake	reviewed	

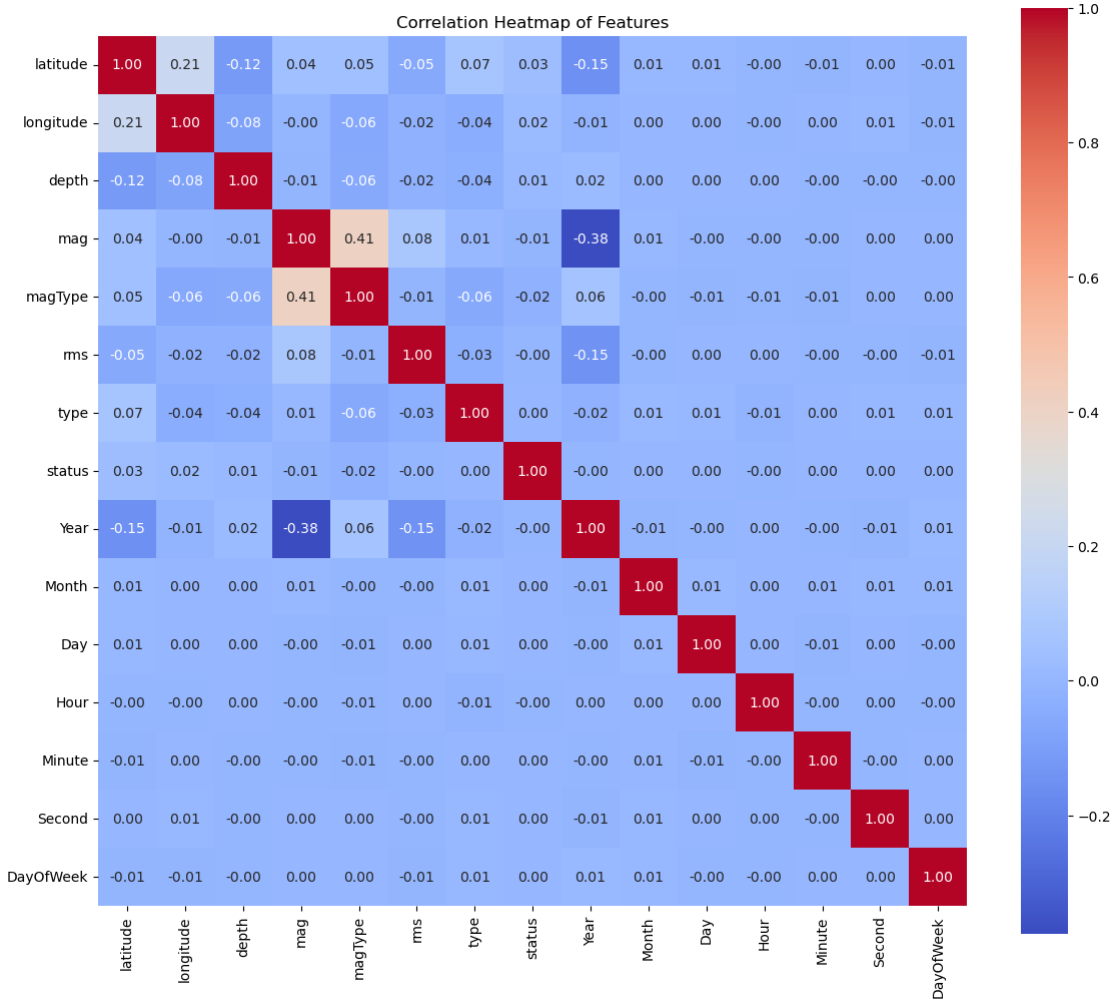
	Year	Month	Day	Hour	Minute	Second	DayOfWeek
16	1904	4	4	10	26	0	0
17	1904	4	4	10	2	34	0
18	1904	6	25	21	0	38	5
19	1904	6	25	14	45	39	5
20	1904	8	30	11	43	20	1

[29]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

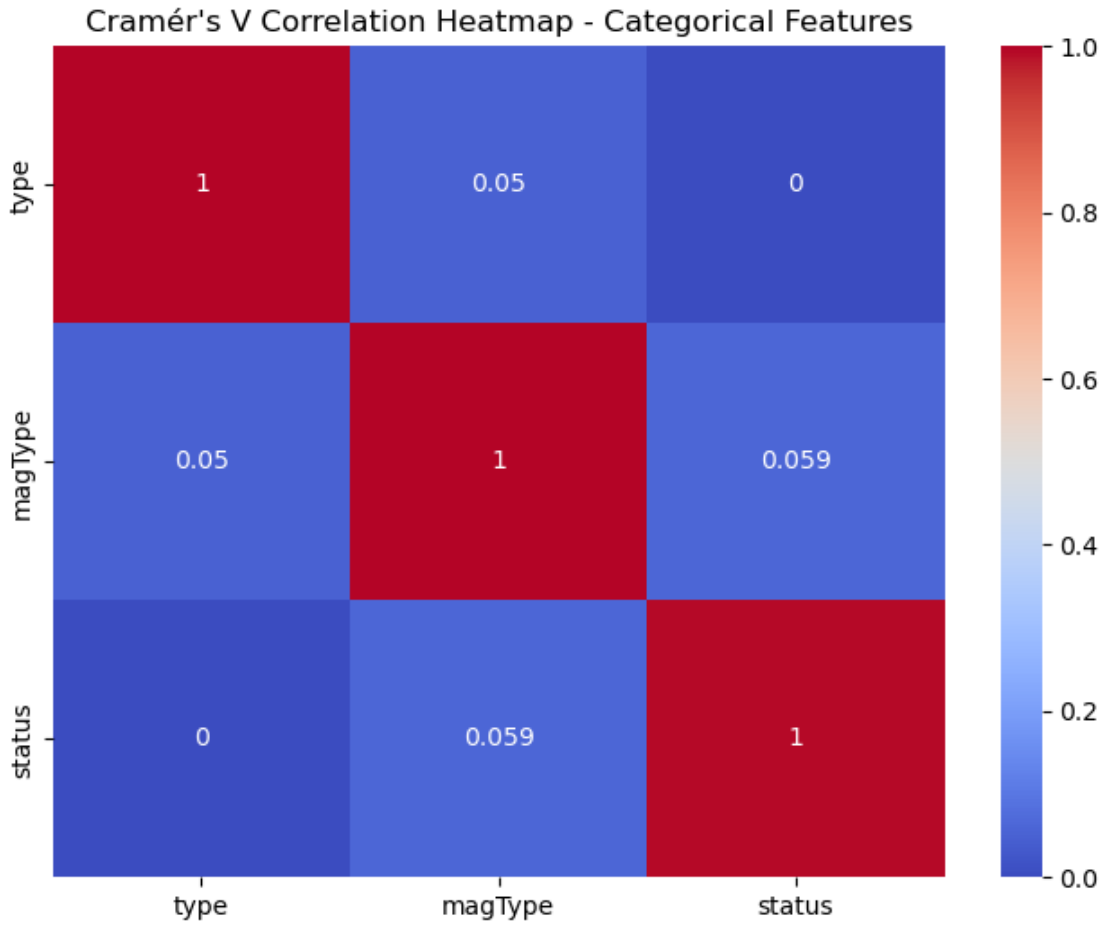
```
Index: 109848 entries, 16 to 110132
Data columns (total 15 columns):
 #   Column      Non-Null Count   Dtype
---  ------      --------------   -----
 0   latitude    109848 non-null  float64
 1   longitude   109848 non-null  float64
 2   depth       109848 non-null  float64
 3   mag         109848 non-null  float64
 4   magType     109848 non-null  object
 5   rms         109848 non-null  float64
 6   type        109848 non-null  object
 7   status      109848 non-null  object
 8   Year        109848 non-null  int32
 9   Month       109848 non-null  int32
 10  Day         109848 non-null  int32
 11  Hour        109848 non-null  int32
 12  Minute      109848 non-null  int32
 13  Second      109848 non-null  int32
 14  DayOfWeek   109848 non-null  int32
dtypes: float64(5), int32(7), object(3)
memory usage: 10.5+ MB
```

[30]:
```python
import seaborn as sns
import matplotlib.pyplot as plt

# List actual categorical columns in the current dataframe
categorical_cols = ['type', 'magType', 'status']  # Only these exist per your␣
 ↪structure

# Create a copy for correlation
df_corr = df.copy()

# Encode categorical columns to numeric codes for correlation calculation
for col in categorical_cols:
    df_corr[col] = df_corr[col].astype('category').cat.codes

# Calculate correlation matrix
corr_matrix = df_corr.corr()

# Plot heatmap
plt.figure(figsize=(14, 12))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', cbar=True,␣
 ↪square=True)
plt.title('Correlation Heatmap of Features')
plt.show()
```



[31]:
```
from scipy.stats import chi2_contingency

# Function to compute Cramér's V correlation between two categorical variables
def cramers_v(x, y):
    confusion_matrix = pd.crosstab(x, y)
    chi2 = chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    phi2 = chi2 / n
    r, k = confusion_matrix.shape
    phi2corr = max(0, phi2 - ((k-1)*(r-1)) / (n-1))
    rcorr = r - ((r-1)**2) / (n-1)
    kcorr = k - ((k-1)**2) / (n-1)
    return np.sqrt(phi2corr / min((kcorr-1), (rcorr-1)))

# Updated list of categorical columns based on current DataFrame
categorical_cols = ['type', 'magType', 'status']
```

```
# Initialize empty DataFrame to store Cramér's V results
cramers_results = pd.DataFrame(np.zeros((len(categorical_cols),␣
 ↪len(categorical_cols))),
                               index=categorical_cols, columns=categorical_cols)

# Calculate Cramér's V for each pair of categorical features
for col1 in categorical_cols:
    for col2 in categorical_cols:
        cramers_results.loc[col1, col2] = cramers_v(df[col1], df[col2])

# Plot heatmap of Cramér's V values
plt.figure(figsize=(8, 6))
sns.heatmap(cramers_results, annot=True, cmap='coolwarm', vmin=0, vmax=1)
plt.title("Cramér's V Correlation Heatmap - Categorical Features")
plt.show()
```



[32]:
```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# List numeric columns present in your DataFrame to scale/normalize
numeric_features = ['latitude', 'longitude', 'depth', 'mag', 'rms', 'year',␣
 ↪'day',
                    'month_sin', 'month_cos', 'hour_sin', 'hour_cos']

# Retain only features that actually exist in df for safety
numeric_features = [col for col in numeric_features if col in df.columns]

# Standard Scaling (z-score normalization)
scaler = StandardScaler()
df[numeric_features] = scaler.fit_transform(df[numeric_features])

# Alternatively, for Min-Max Normalization (uncomment below)
# normalizer = MinMaxScaler()
# df[numeric_features] = normalizer.fit_transform(df[numeric_features])

print(f"Scaled numeric features: {numeric_features}")
```

```
Scaled numeric features: ['latitude', 'longitude', 'depth', 'mag', 'rms']
```

[33]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 109848 entries, 16 to 110132
Data columns (total 15 columns):
 #   Column     Non-Null Count   Dtype
---  ------     --------------   -----
 0   latitude   109848 non-null  float64
 1   longitude  109848 non-null  float64
 2   depth      109848 non-null  float64
 3   mag        109848 non-null  float64
 4   magType    109848 non-null  object
 5   rms        109848 non-null  float64
 6   type       109848 non-null  object
 7   status     109848 non-null  object
 8   Year       109848 non-null  int32
 9   Month      109848 non-null  int32
 10  Day        109848 non-null  int32
 11  Hour       109848 non-null  int32
 12  Minute     109848 non-null  int32
 13  Second     109848 non-null  int32
 14  DayOfWeek  109848 non-null  int32
dtypes: float64(5), int32(7), object(3)
memory usage: 10.5+ MB
```

[34]:
```
# Save the preprocessed dataframe to a new CSV file
df.to_csv('preprocessed_earthquake_data.csv', index=False)
```

Observations:

1.We first analyzed missing values and calculated their percentage for each column to understand data quality.

2.Columns having more than 30% missing values were removed to improve dataset reliability and consistency.

3.We dropped rows containing missing values in the depth column to ensure accurate and meaningful analysis.

4.We deleted unnecessary or ID-like columns such as Unnamed: 0, id, net, locationSource, and magSource as they were not useful for analysis.

5.The dataset was cleaned and reshaped after these preprocessing steps to retain only the most relevant features.

6.Linear regression imputation was likely used to fill missing values for certain numeric features (e.g., rms) based on correlated attributes.

7.After cleaning, most numerical variables showed weak or minimal correlation, suggesting low linear relationships between them.

8.A moderate positive correlation was observed between magnitude (mag) and magType, while Year showed a slight negative correlation with depth.

9.Categorical features such as type, magType, and status showed weak relationships with other variables, indicating they are mostly independent.

10.The final preprocessed dataset became more balanced, consistent, and ready for further exploratory analysis or model building.